

DAS User Manual

Group 47

1 How the Code Functions

1.1 Defining the Functions

1.1.1 Importing Libraries

Firstly, the libraries needed for the code to function are imported. The 'import board' is needed in order to use the raspberry pi pico board. The 'pwmio' stands for pulse width modulation input output. 'time' is used to keep track of variables. 'analogio' is analogue in out, and is needed to use analogue components. Similarly, 'digitalio' is digital in out. 'busio' is imported for bus connections, such as wires, and is needed to initialise pins. 'sdcardio' is used for sd card input and output and 'storage' is needed alongside this to initialise the file system for the sd card. 'os' imports the os to find the file tree.

1.1.2 Creating the Functions

The first function is then defined, which simply converts counts to voltage using the equation:

$$voltage = \frac{counts}{2^{bitDepth}-1} * (V_{max} - V_{min}) + V_{min}$$

A similar function is defined which converts the voltage we just calculated into a measurement. For this, the following equation is used:

$$measurement = \frac{(SensorVoltage - SensorVoltage_{min})(Sensor_{max} - Sensor_{min})}{SensorVoltage_{max} - SensorVoltage_{min}} + Sensor_{min}$$

The following three functions calculate motor voltage, motor current and motor RPM from the analogue voltage, current and RPM, respectively.

For Motor Voltage:

V Max	V Min	Sensor Max	Sensor Min	Sensor V Max	Sensor V Min
3.3	0	2.91	0.2	24.9	1.79

The sensor counts are equal to the analogue voltage value. The sensor voltage is then calculated by inputting the sensor counts and the maximum and minimum voltage using 'counts to volts'.

The measurement is calculated by inputting the sensor voltage, the maximum and minimum sensor voltage and the maximum sensor value using 'voltage to measurement'. This measurement is equal to the motor voltage, which is returned. The values used are shown above.

For Motor Current:

The sensor counts is again equal to the analogue current value. The sensor voltage is again found with the 'counts to volts' function by inputting the sensor counts, the maximum voltage and the

I Max	I Min	V Max	V Min	V Bias	Current Sensitivity
40	0	3.3	0	1.62	25 A/V

minimum voltage. Then, in order to obtain the maximum and minimum sensor voltage, we must use the following equation:

$$SV_{max} = \frac{I_{max}}{CurrentSensitivity} + V_{bias}$$

$$SV_{min} = \frac{I_{min}}{CurrentSensitivity} + V_{bias}$$

The current is then calculated using the 'voltage to measurement' function, inputting sensor voltage, maximum and minimum SV and maximum and minimum current.

For Motor RPM:

RPM Max	RPM Min	V Max	V Min	V Bias	RPM Sensitivity
12000	0	5	0	0	3636

The sensor counts in this function is the RPM analog value. The sensor voltage is once again calculated using the 'counts to volts' function, by inputting sensor counts and the maximum and minimum voltage. The maximum and minimum sensor voltage are calculated using:

$$SV_{max} = \frac{RPM_{max}}{RPMsensitivity} + V_{bias}$$

$$SV_{min} = \frac{RPM_{min}}{RPMsensitivity} + V_{bias}$$

The RPM is calculated using the 'voltage to measurement' function, which needs the inputs of sensor voltage, the maximum and minimum sensor voltage and the maximum and minimum RPM.

Next, a function called 'LoadCellOutput' is defined. The input to this function is the load cell UART connection. The function reads the transmitted signal and decodes it using ascii. Then, it splits this signal into three subsequent parts which are the frame identifier, thrust and torque. The decoded signal is received in the <AA,BB.BBB,CC.CCC> format, where the three arrays AA, BB.BBB and CC.CCC are the frame identifier, thrust and torque, respectively. The line print(e) is for debugging and is hence currently commented out for normal operation. If the code breaks, the user can uncomment this line in order to identify the error. The 'try' loop within this function means that the code will try to decode the data no matter what it is however if there is any error, the 'except' statement will ensure simply that [0,0] is returned. This means that if the UART message is paused or corrupted, the system will not crash.

motorThrust and motorTorque are then defined as functions, which take the desired data (the array of thrust and torque) and split it into the relevant respective parts.

Another function is defined, named calcPulseWidth. This simply calculates the pulse width from the throttle values. We need this to be between 1000-2000 microseconds, therefore $pw_{min} = 1000$ and $pw_{max} = 2000$. We also used $throttle_{min} = 0$ and $throttle_{max} = 100$:

$$pulseWidth = \frac{(throttle - throttle_{min})(pw_{max} - pw_{min})}{throttle_{max} - throttle_{min}} + pw_{min}$$

Next, a function to calculate the duty for the ESC is created. This uses pulse width which we have calculated above, and bit depth which is a fixed value of 16 for the pico board:

$$duty = \frac{pulseWidth}{1000000} * motorSignal * (2^{bitDepth} - 1)$$

Again, for use later on, a function to average values is made. This averages the input values, which can be specified when the function is called upon.

A function is then created to sort through files and name them appropriately. This function, named 'findFileNum' uses the os library and the 'listdir' function to collect all the files in a given directory. Then, 'file.sort' sorts the files in order from zero to the most recent file number. 'files' is the name given to all the files in the directory which is defined earlier. 'file' is the attribute of a file (which can be found in 'files'). A 'for' loop is created, which checks if a file starts with DAS. It then splits off the hyphen and creates an array out of the two halves. After finding the second item in the array (the number), it splits off the '.csv' to leave the number. This is then incremented up by 1.

For example, if a file was called "DAS-9.csv", the programme would split it in the following steps:

1. DAS-9.csv
2. [DAS 9.csv]
3. 9.csv
4. 9

This number (in this example, 9) is then incremented up by one, to become 10. This new file number is returned to the program.

1.2 The Main 'if' Condition

1.2.1 The Basis

Now all functions are defined, a main 'if' condition is made. The condition that if `__name__=="__main__"` is there so that the file will only run if it is run directly (not through any other calls).

The pin connections are then initialised, with the voltage sensor in A1, the current sensor in A2 and the RPM sensor in A3. The load cell is also initialised for UART serial receiving. The transmitter, Tx, is set to GP12 and the receiver, Rx, is set to GP13. The baud rate is set to 9600 and the timeout to 0.05s. This timeout value ensures a full frame is always received.

1.2.2 Indicators

Afterwards, the LED is initialised, which is used for a visual airspeed change. This is connected to GP5 and its direction is set to an output. A switch is implemented which allows you to un-pause after a change in throttle. This is connected via GP22. An emergency stop is also implemented via GP15 which stops all processes and terminated the loop. This line of code will be copied into each loop of the whole program to ensure that the code can be stopped at any time, if needed. A final indicator in the form of a buzzer is added. This uses a variable frequency and signals warnings related to voltage and current. The voltage warning is set to 2000 Hz and the current warning is set to 500 Hz. The buzzer duty here is set to half the bit depth, 32768. More on the buzzer will be defined later, such as the conditions for when the buzzer should sound.

1.2.3 Constants and Timers

The next 8 lines in the code initialise constants needed and set timers:

- The sample number is initialised at zero and increments for each sample taken.
- The bit depth is a constant number for the Pico, which is 16.
- The motor signal is set at 50 Hz, which is the frequency of the signal the ESC will receive.
- `measurementTimer` and `measurementUpdate` initialises the timer for long should be between each measurement and hence takes a measurement every second (as `measurementUpdate = 1`).
- The stall time is set to 2, which allows 2 seconds of pausing between throttle changes.
- `throttleSampleTimer` and `throttleSampleTimer` initialises the timer and sets the time samples will be taken for each throttle speed. Here, it is set to 10, therefore the sample will be taken every 10 seconds.
- `averageTimer` and `averageWait` initialises a timer and for each set of measurements, it waits 5 seconds before it takes the average, as `averageWait` is set to 5. This allows the average to always be over 5 measurements.
- Finally, the PWM signal is initialised for the electronic speed controller. It is set to duty cycle, which is zero as before testing begins, the throttle should be zero.

The measurement variables are now initialised, with thrust, torque, RPM, current and voltage all set to [0, 0, 0, 0, 0]. The only exception is airspeed, which is set to [8, 10, 12] for the 3 different airspeed values we use. `i` is set as an index which starts at zero.

The variable `highCurrentTime` is set to 10. This is used to check if the current has been above 20 amps for more than 10 seconds, which again will be used later.

An array called `throttle` is made which is [10, 20, 30, 40, 50, 60, 70, 80] and allows the change between throttles of 10 to 80 as specified by requirements.

A flag called `sampling` is set to `False`, which is used at the end to break the loop once sampling has been completed.

`highCurrent` is initialised to zero, which is a variable used for the timers. The timer will only activate once the current is above 20 amps.

Another flag is created called `currentThreshold`, and is set to `False`. This determines if the current is above the maximum threshold of 20 amps.

1.2.4 SD Card

Within the same 'if' loop as expressed before, we set up the sd card and all related files. This sd card will store the data, which will shortly be defined.

First, the sd card needs to be initialised. The serial peripheral interface is introduced, which creates the interface between the pico and the sd card. The `cs` is also defined (in GP17). The sd card is initialised using the `spi` and `cs` as defined above. In order to operate, the virtual file system

needs to be set up, shown as Vfs. We also need to mount the sd card to make the files available for the user to access, which uses the function 'storage.mount'.

If there are no previous files, the first one to be made will have the number zero. The code then opens a new file with an incremented file name. It does this by using an f string, which allows variables to be directly inputted into the string. 'w' is also used to indicate we are writing a file. Again, this will create a new file if the file we are trying to write to doesn't exist.

The next line, using 'file.write' writes the headings of the document. In order from left to right in the file, these will be; Sample Number, Wind Speed (ms^{-1}), ESC Throttle Setting (Hz), Average Current (A), Average Voltage (V), Average RPM (RPM), Average Force (N), Average Torque (Nm).

1.2.5 The Inner Loops

Another 'while' loop is created, that states that while the throttle is not off and the sampling isn't complete, k in range (0,3) goes through the air speeds and it will pause the motor to allow airspeed change. This will resultantly turn on the LED. An 'if' clause is added after, which is the emergency stop. This states if the throttle is zero, break; this breaks out the loop. In this same 'k in range' loop, the LED turns off when the airspeed has changed and resumes sampling.

Another inner loop is made here, which goes through the throttle values using the j index indicator. It prints to the user that it is pausing due to invalid data, allowing 2 seconds of stall between throttle changes. Another check for the emergency button is implemented into this loop. The throttle is inputted to calculate pulse width, which then calculates duty cycle. The duty cycle is used by the ESC to change the RPM. After changing, there is a 2 second stall. During this stall time, the values for RPM, current and voltage are reset.

A sample timer and average timer are then started and another inner loop is created, which loops for the duration of 10 seconds. Another emergency stop check is used here. If the timer is larger than the measurement timer + the update measurement, the sensor data is retrieved and the current variables are measured. Torque and thrust are initialised and forced into a loop. This loop retrieves the UART message and decodes it. motorThrust and motorTorque are used to retrieve said thrust and torques. If there is an error (such as if the thrust and torque remain equal to zero), it will stay in the loop and keep trying to receive a valid UART message. The 'while' loop used prevents crashing in the case of any error. After another emergency stop check, the RPM, current and voltage values are retrieved from their respective functions and are stored in an array. Power is then calculated and the desired air speed is retrieved. This is used for the serial message output.

1.2.6 Buzzer Checks

A check is made to check if the voltage and current are within their respective boundaries. This is where the buzzer functions are used. If they remain in their boundaries, the buzzer will not sound. The high current timer is set to an arbitrarily high value to prevent unwanted buzzing due to current warning. If this happens and the current was previously above 20 amps, the current threshold is reset.

If the voltage is less than 22V, the buzzer is set off. If the current is greater than 20 amps and the current threshold has not been broken before, a timer is activated which will not be reset on loop

iterations. If the current drops below 20 amps, the timer becomes arbitrarily large such that the buzzer will not go off. The timer checks that if the circuit has had high current for more than 10 seconds, it will buzz. Otherwise, the value is set to zero.

1.2.7 Writing the Files

After 5 seconds, the averages of numerous values are made and collected. 'file.write' is used to allow the overwriting of any value older than 5 seconds. The file is opened, using 'a' to signify we are appending. The sample number is incremented and the code writes to the file:

- Sample Number as an integer
- Desired Air Speed as an integer
- Pulse Width as a float with zero decimal places
- Average Current as a float with 1 decimal place
- Average Voltage as a float with 1 decimal place
- Average RPM as a float with zero decimal places
- Average Thrust as a float with 3 decimal places
- Average Torque as a float with 3 decimal places

A serial message is also printed for the user, displaying the current air speed, desired airspeed, current ESC throttle, latest force, thrust, latest torque, latest RPM and latest power.

1.2.8 Complete Sampling

After a complete sampling is done, i is incremented until it reaches 5. Then it resets back to zero and will continue sampling, overwriting the previous values. This creates a moving average for each measurement where each average is taken over a 5 second period.

Once sampling has completed for every throttle value for each air speed, sampling is set to True which will break the main loop.

The throttle value on the turbine will be turned off after sampling is complete (when the duty cycle is zero). If the throttle is forced to zero, the phrase "Emergency Stop Initiated" will be printed. Otherwise, the phrase "Data successfully saved to: [file location]" will be printed.

2 The Outputted File

The code will output a '.csv' file with the following columns:

- An incrementing sample number per test point
- Windspeed
- ESC Throttle Setting (as pulse width)
- Average Current
- Average Velocity
- Average RPM
- Average Force
- Average Torque

One line is written per matrix test point and a column header line is included at the start of the csv file.

3 User Interactions

Action	When to Act	Outcome
Push Button Left (away from sd card)	When LED comes on	Resumes the process after an airspeed change
Push Button Right (towards sd card)	Whenever deemed necessary	Stops the whole operation and breaks the codes main loop

Alarm Mechanism	Reason	User Outcome
LED turns on	Process is paused for change in airspeed	Press the button left in order to resume
Buzzer sounds at 2000 Hz	The voltage is less than 22V	Voltage needs to be increased
Buzzer sounds at 500 Hz	The current is above 20 A	Current needs to be decreased

The user will also be provided with a serial message interface. The monitoring message contains:

- Current expected airspeed setting
- Current ESC throttle setting (as a percentage)
- Latest force measurement
- Latest torque measurement
- Latest RPM
- Latest Power in Watts

4 Circuit Diagram

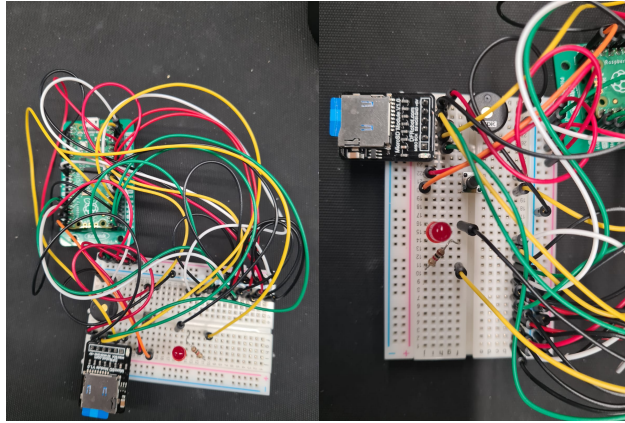


Figure 1: Circuit Diagram

Figure 2: View of Breadboard

Pins 1 through 14 are connected to the breadboard in a1 through a14, respectively.

A ground rail is set up on the negative power strip and a 3.3V rail is set up on the positive power strip.

Pin 1 - Ground for the RPM sensor, which is connected to the ground power strip

Pin 2 - RPM sensor supplied with 5.5V. This is connected to VBUS on the pico board.

Pin 3 - RPM sensor output. This is connected to A0 (GP26) on the pico.

Pin 4 - Ground for the voltage sensor. This is connected to the ground power strip

Pin 5 - Voltage sensor output. This is connected to A1 (GP27) on the pico.

Pin 6 - Ground for the current sensor. This is connected to the ground power strip.

Pin 7 - Current sensor power. This is connected to the positive power strip.

Pin 8 - Current sensor output. This is connected to A2 (GP28) on the pico.

Pin 9 - Ground for the load cell UART message. This is connected to the negative power strip.

Pin 10 - UART message is given to GP13 on the pico, which is a receiving pin

Pin 11 - Connected to GP12 on the pico which is a transmitter pin. This is needed to complete the circuit but is not used in the code.

Pin 12 - Ground for the electronic speed controller (ESC). This is connected to the negative power strip.

Pin 13 - Connected to VSYS as ESC provides a regulated power supply output at 5V.

Pin 14 - Connected to GP20 on the pico. This controls signal input to change the RPM of the motor.

LED - Connected through a ground (to the negative power rail), a resistor and then to GP5 on the pico.

Buzzer - Connected through a ground (to the negative power rail), and then to GP7 on the pico.

Button - Each side there is a wire which supplies the button with 3.3V. On one side, a wire controls the airspeed change and is connected to GP22. On the other side, the emergency stop is controlled and is connected to GP15.

SD Card - MISO is connected to GP16. SCK is connected to GP18. CS is connected to GP17. MOSI is connected to GP19. the GND (ground) is connected to the ground on the pico between GP17 and GP18. The 3.3V power is connected to the positive power strip on the breadboard.